

Quick sort **HOARE** < 10

9 5 7 4 2 6 | 10 19 12 18 11 21

↓

2 4 5 6 7 9 11 12 18 19 21 100

è un caso che mi ha ritardato con questa suddivisione, ma posso lavorare per ricreare

10 19 9 5 12 18 7 4 11 2 6 21

Prendo il 1° valore

10 ↓ ↓ ↓ ↓ ↓ 18 12 19 10
19 9 5 12 18 7 4 11 2 6 21
2 1 7 ↑ ↑ √ ↑ ↑

6 < 10 gli scambia

6 2 9 5 4 7 | 18 12 11 19 10 21

Cioè un numero della prima parte è minore di quello della seconda, continuo col ordinare ricorrendo le 2 metà

2 4 5 6 7 9 10 11 12 18 19 21

Il "miglior" numero di "intronzo" è la mediana

Il quicksort non è un algoritmo fisso ma una famiglia di algoritmi

Il libro quick sort **lento**

procedure QS (A, l, r)

QS ($A, l, A.length$)

if $l < r$ then
 cut = PARTITION (A, l, r)

QS (A, l, cut)

QS ($A, cut+1, r$)

procedure PARTITION (A, l, r)

pivot = $A[l]$

$jx = l - 1$

$ox = r + 1$

while TRUE do

do

$jx = jx + 1$

while $A[jx] < \text{pivot}$

do

$ox = ox - 1$

while $A[ox] > \text{pivot}$

if $jx < ox$ then

 exchange $A[jx]$ & $A[ox]$

else

 return ox

$\downarrow$ 4 1 5 2 7 9 $\text{pivot} = 4$
 $\uparrow$ ok
 $QS(A, 1, 6)$
 $P(A, 1, 6)$

$\downarrow$ ④
~~4~~ 1 5 ~~2~~ 7 9
 $\uparrow$ ④
 2

2 1 | 5 4 7 9 $\text{cut} = 2$
 $\uparrow$ ④

$QS(A, 1, 2)$ $\text{pivot} = 1$
 $\text{cut} = 1$ $2 \times$ $\downarrow$ 1 2
 $QS(A, 1, 1)$ $\text{case lpe, ho finito}$ $\uparrow$ ok
 1^a parte

$QS(A, 3, 6)$
 $P(A, 3, 6)$
 $\text{pivot} = 5$
 $\text{cut} = 3$

$\downarrow$ ④
~~5~~ ~~4~~ 7 9
 $\uparrow$ ④
 5

$QS(A, 4, 6)$ $2 \times$
 $\downarrow$ 5 | 7 9
 $P(A, 4, 6)$ $\uparrow$ ok

$\text{pivot} = 5$

$\text{cut} = 5$

$QS(A, 5)$

Empire position

1) $\text{pivot} = 12$

12	12	15	20	13	10
↓	↓				↑
12	12				ok

2) $\text{pivot} = 15$

15	14	10	12	8	15
↓					↑
15					ok

3) $\text{pivot} = 15$

15	20	18	25	16	15
↓					↓
15					ok

4) $\text{pivot} = 15$

15	12	10	16	16	18	15	20
↓			↓	↓		↓	↑
15			5	16	18	8	ok

Dimostrazione correttezza

procedure QS(A, l, r)

if $l < r$ then
 cut = PARTITION(A, l, r)

 QS(A, l, cut)
 QS(A, cut+1, r)

QS(A, 1, A.length)

QS corretta su input di dimensione $n+1$

→ corretta per n

→ l , cut e cut+1, r sono più piccoli di l, r

$$A[l, \text{cut}] \leq A[\text{cut}+1, r]$$

procedure PARTITION(A, l, r)

pivot = A[l]

$x = l - 1$
 $ox = r + 1$

while TRUE do

 do

$x = x + 1$

 while $A[x] < \text{pivot}$

 do

$ox = ox - 1$

 while $A[ox] > \text{pivot}$

 if $x < ox$ then

 scambia $A[x]$ e $A[ox]$

 else

 return ox

x e ox non escono al di fuori di l e r

- quando accade $A[x] \geq \text{pivot}$

- quando accade $A[ox] \leq \text{pivot}$

- $l \leq x \leq ox \leq r$

$x = ox \Rightarrow OK$ subito

$x < ox \Rightarrow$

INV. $A[l \dots x] \leq A[ox \dots r]$

$\forall i, l \leq i \leq x \wedge ox \leq j \leq r$

$\Rightarrow A[i] \leq A[j]$

Al termine di PARTITION ho $l \leq ox \leq r$

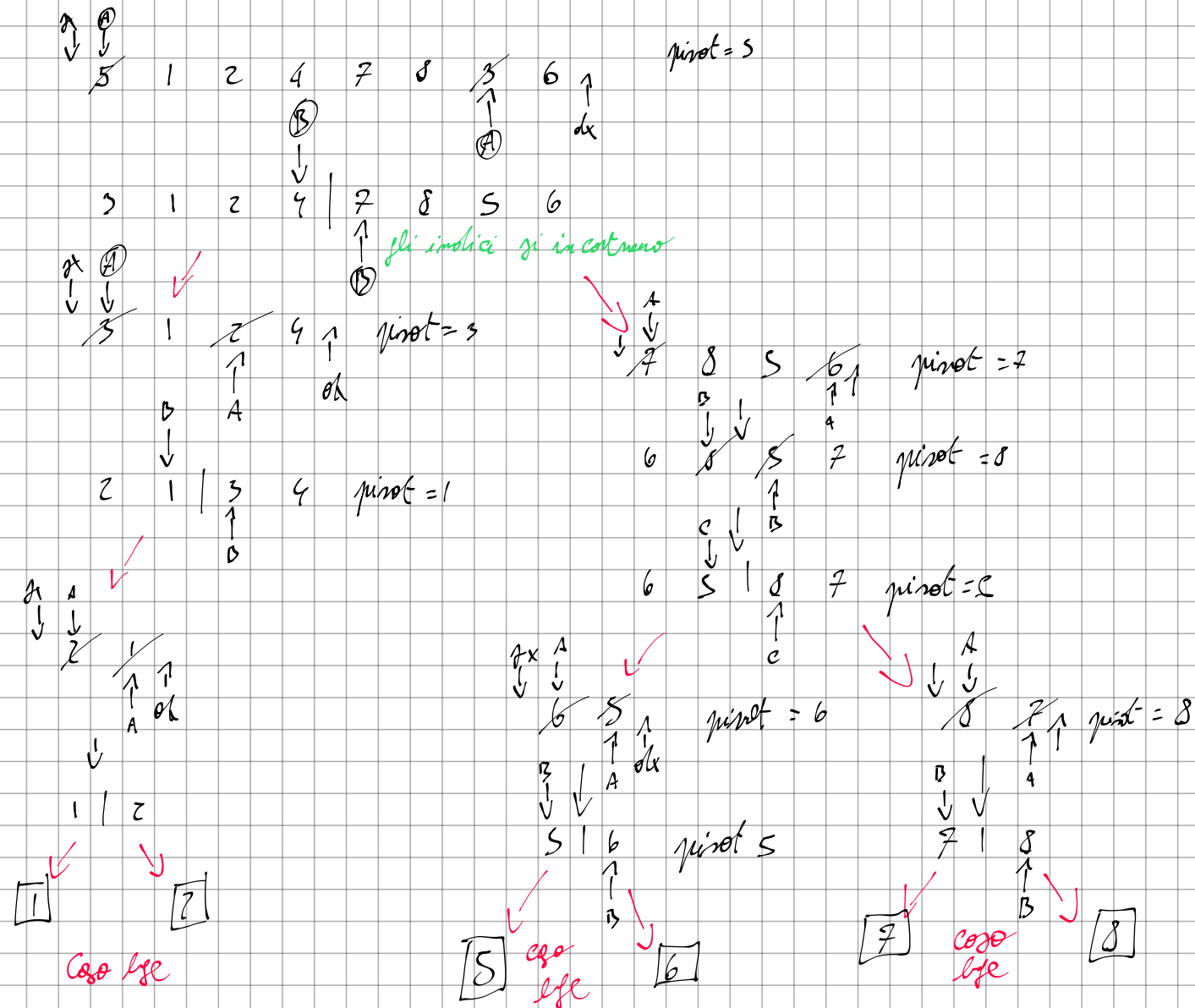
alla fine dei primi 2 do while

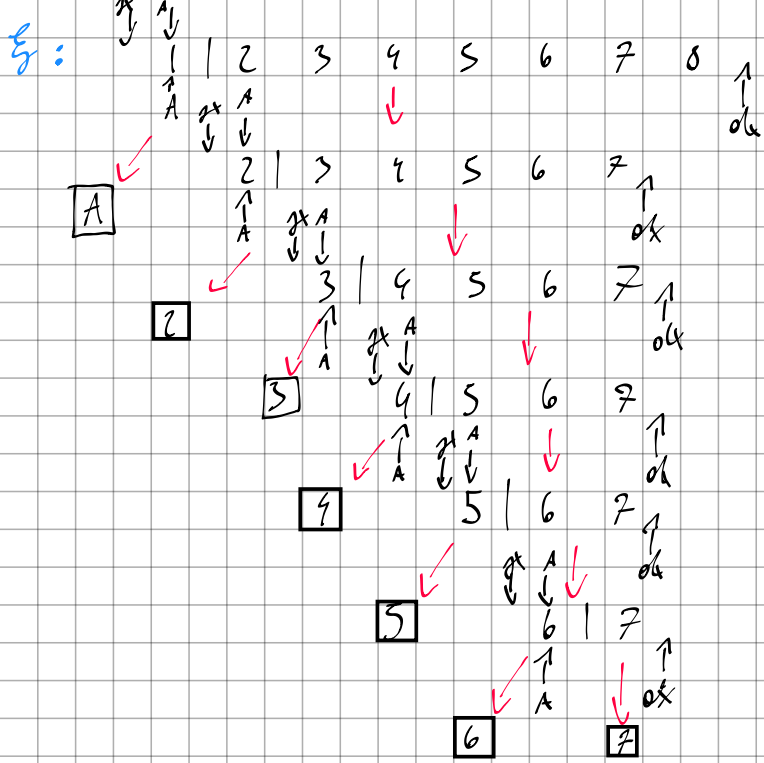
$ox \leq r \Rightarrow OK$

Algoritmo di Strogan

Algoritmo divide et impera utilizzato per calcolare il prodotto riga x colonna di 2 matrici, è un $\Theta(n^2.8)$

5





Il quick sort non è in-place

tempi di calcolo

procedure PARTITION (A, l, r)

$pivot = A[l]$

$ix = l - 1$

$ox = r + 1$

 while TRUE do

 do

 | $ix = ix + 1$

 while $A[ix] < pivot$

 do

 | $ox = ox - 1$

 while $A[ox] > pivot$

 if $ix < ox$ then

 | scambio $A[ix]$ e $A[ox]$

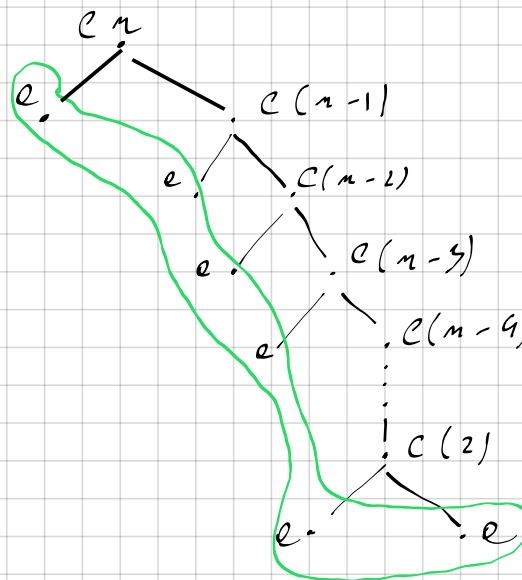
 else

 | return ox

tutti gli elementi vengono visiti almeno una volta

$$T(n) = \Theta(n) + \Theta(1) + T(n-1)$$

non si può applicare il teorema dell'agente, occorre usare l'albero di ricorsione



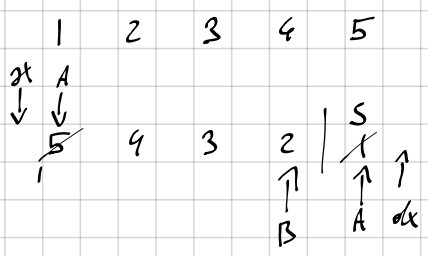
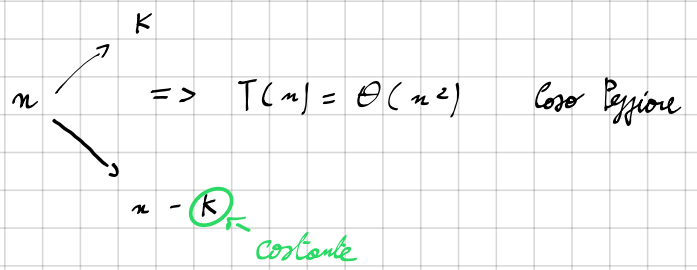
$$\# \text{ foglie} = n$$

$$t_{\text{foglie}} = \Theta(n)$$

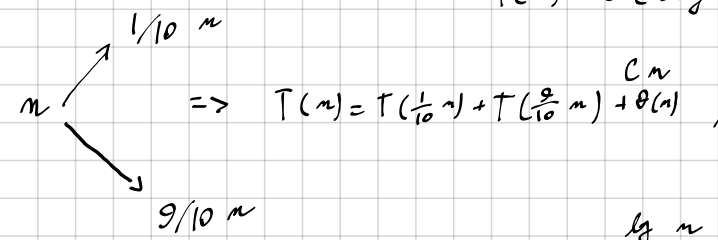
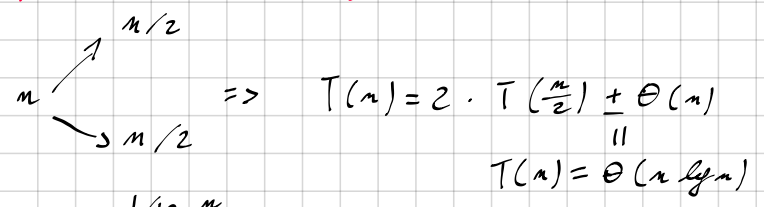
$$t_{\text{nodi interni}} = c \sum_{i=2}^n i = c \cdot \left[\frac{n \cdot (n+1)}{2} - 1 \right] = \Theta(n^2)$$

$$T(n) = t_{\text{foglie}} + t_{\text{interni}} = \Theta(n) + \Theta(n^2) = \Theta(n^2)$$

Partizionamento bilanciato



Partizionamento bilanciato

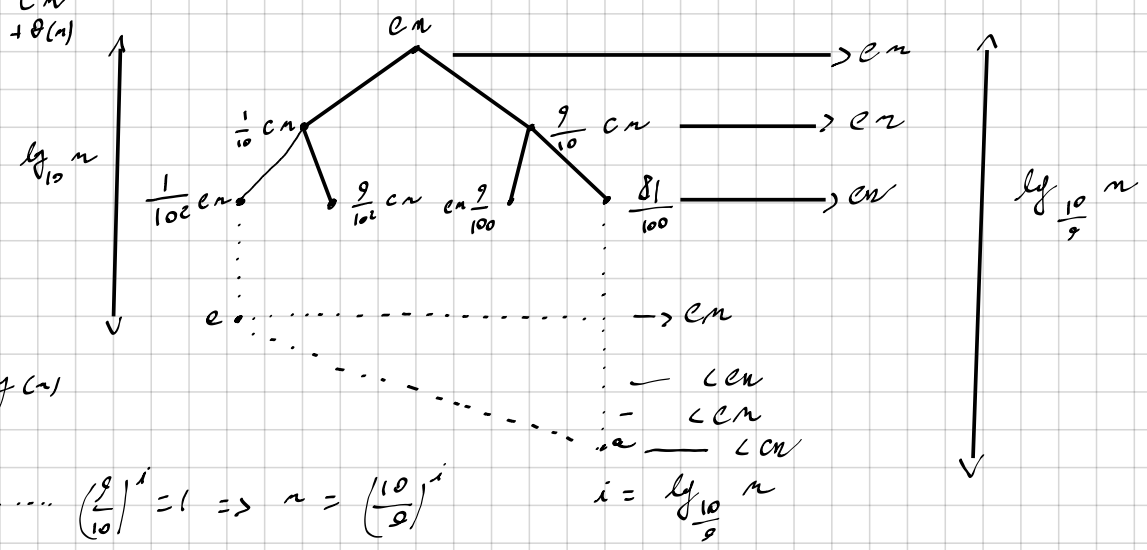


$1 n \rightarrow \frac{1}{10} n \rightarrow \frac{1}{10^2} n \rightarrow \frac{1}{10^3} n$

$\frac{1}{10^i} n = 1 \Rightarrow n = 10^i \quad i = n \lg(10)$

$1 n \rightarrow \frac{9}{10} n \rightarrow \left(\frac{9}{10}\right)^2 n \rightarrow \left(\frac{9}{10}\right)^3 n \dots \left(\frac{9}{10}\right)^i = 1 \Rightarrow n = \left(\frac{10}{9}\right)^i$

Caso migliore



$$T(n) \leq \lg \frac{10}{9} n \cdot c \cdot n = O(n \lg n)$$

$$T(n) \geq \lg_{10} n \cdot c \cdot n = \Omega(n \lg n)$$

$$T(n) = \Theta(n \lg n)$$

$$n \begin{cases} \alpha n \\ (1-\alpha)n \end{cases} \Rightarrow T(n) = T\left(\frac{1}{10}n\right) + T\left(\frac{9}{10}n\right) + \Theta(n) = \Theta(n \lg n)$$

Se partition il 10% delle volte mi da una split $\rightarrow$ bilanciato $\frac{1}{10}n, \frac{9}{10}n$
 " " il 20% " " " " " " partition $\rightarrow$ bilanciamento di $\frac{1}{10}n, \frac{9}{10}n$

Caso medio

Si alternano n p bilanciati con p sbilanciati $\Rightarrow T(n) = \Theta(n \lg n)$

Come evitare il caso peggiore

In caso ottengo un array ordinato lo mischio in modo random per riportarlo al caso medio

Ma disordinare un array è costoso, quindi conviene scegliere un pivot e caso per caso scegliere di nuovo il primo elemento

PSEUDOCODE - Quick Sort Random

procedure RND_PARTITION(A, l, r)

 sceglie $A[l]$ con un elemento $A[i]$
 con i galto a caso in $[l, r]$

 return PARTITION(A, l, r)

tempo Oltro RND_QS = $\Theta(n \lg n)$, rimane in-place ma non è stabile

RND_QS(A, l, r)

if $l < r$ then

 cut = RND_PARTITION(A, l, r)
 RND_QS(A, l, cut)
 RND_QS($A, cut+1, r$)

oppure

RND_QS(A, l, r)

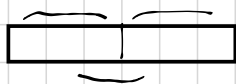
while $l < r$ do

 cut = RND_PARTITION(A, l, r)
 RND_QS(A, l, cut)
 RND_QS($A, cut+1, r$)

Conte 000

Input: array A

Output: quante volte compare 000



ma posso avere



procedure CONTA000DI (A, l, r)

logi base

if $(r - l + 1 < 3)$ then *se ho meno di 3 elementi
non ci può essere una tripletta*
return 0

if $r - l + 1 = 3$ then

if $(A[l] = 0 \ \&\& \ A[l+1] = 0 \ \&\& \ A[l+2] = 0)$ then
return 1

else
return 0

mid = $\frac{l+r}{2}$ *DIVIDE*

esx = conte000DI(A, l, mid)

colx = conte000DI(A, mid+1, r)

cx = 0

if $A[mid-1] = 0 \ \&\& \ A[mid] = 0 \ \&\& \ A[mid+1] = 0$ then
cx = cx + 1

if $A[mid] = 0 \ \&\& \ A[mid+1] = 0 \ \&\& \ A[mid+2] = 0$ then
cx = cx + 1

return esx + cx + colx

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(1)$$

$$n \geq 1$$

$$l > 1$$

$$f(n) > 0$$

$$n^{\log_b a} = n$$

$$\begin{aligned} \theta(1) &\rightarrow \theta(n^{\epsilon}) \checkmark \text{ esempio } \epsilon = 1/2 \\ &\rightarrow \theta(n) \times \\ &\rightarrow \Omega(n^{\epsilon+1}) \times \end{aligned}$$

Array unimodulare

Input: array A unimodulare

Output: posizione del minimo

$$T(n) = O(\lg(n))$$

procedura UNIMODDI(A, l, r)

base case

if $l = r$ then

return l

mid = $\lfloor \frac{l+r}{2} \rfloor$ DIVIDE

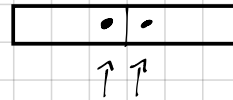
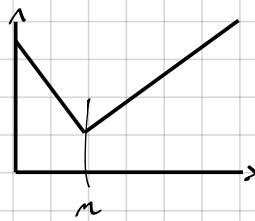
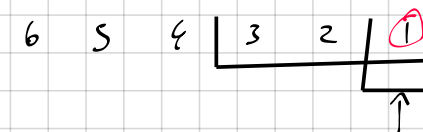
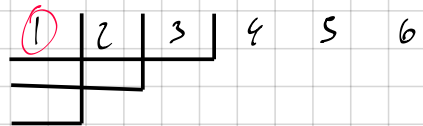
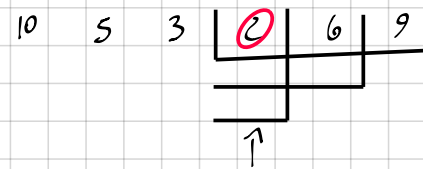
if $A[mid] < A[mid+1]$ then

return (A, l, mid)

else

return UNIMODDI(A, mid+1, r)

IMPERF



controllo, se $x > a$ vedo a destra punto faccio il contrario

$$T(n) = T(\frac{n}{2}) + O(1)$$

$$n = 1 \geq 1$$

$$l = r \geq 1$$

$$O(1) \geq 0$$

$$n^{\lg_2 a} = n^{\lg_2 1} = n^0 = 1$$

$$\Theta(1) \rightarrow O(n^{\theta - \epsilon})$$

$$\Theta(n^0) \rightarrow T(n) = \Theta(n) \cdot \lg(n) = \lg(n)$$

$$\Omega(n^{0 + \epsilon})$$